1. True/False Questions

Problem 1.1 False. The forget gate is the only gate that scales the old state c{t-1}. The input gate and output gate act on the new candidate and the output.

Problem 1.2 True. Each head uses size d/h, so one head costs $O(n^2 \cdot d/h)$. With h heads, the total is $h \times O(n^2 \cdot d/h) = O(n^2 d)$. The cost stays the same.

Problem 1.3 False. BPE is bottom-up. Also BERT does not use BPE.

Problem 1.4 True. Attention does not know word order by itself. We have to add a positional embedding to each token tells the model where the token is.

Problem 1.5 False. In the decoder's second attention block (cross-attention), the encoder output gives the Key and Value. The Query comes from the decoder (slides 16-2 and 16-3).

Problem 1.6 True. Tokenizers split text into subwords, so one word can be several tokens.

Problem 1.7 False. In decoding, only the new token needs Q, K, V. Each projection costs $O(d^2)$, so the cost is $O(d^2)$.

Problem 1.8 True. FlashAttention tiles the computation so it stays in fast on-chip memory (SRAM). This reduces slow memory reads and writes.

Problem 1.9 True. Humans rank or rate outputs, a reward model learns from that, and the reward model then guides the LLM's training.

Problem 1.10 False. KV-cache is used in inference (decoding). In training, all tokens are processed in parallel, so there is no need to cache K and V.

2 Lab (1): Recurrent Neural Network for Sentiment Analysis (35pts)

(a)

```
def load_imdb(base_csv: str = 'IMDBDataset.csv'):
    hp = HyperParams()
    df = pd.read_csv(base_csv)

    def clean_text(text):
        text = re.sub(r'<.*?>', '', text)  # remove HTML tags
```

```
        return text.lower().strip()

    df['review'] = df['review'].apply(clean_text)
    X = df['review'].to_numpy()
    y = df['sentiment'].to_numpy()

    # 20% test; then 12.5% of the remaining 80% = 10% validation, 70% train
    X_train_val, X_test, y_train_val, y_test = train_test_split(
        X, y, test_size=0.20, random_state=hp.SEED, stratify=y)
    X_train, X_val, y_train, y_val = train_test_split(
        X_train_val, y_train_val, test_size=0.125,
        random_state=hp.SEED, stratify=y_train_val)

    print(f'shape of train data is {X_train.shape}')
    print(f'shape of test data is {X_test.shape}')
    print(f'shape of valid data is {X_val.shape}')
    return X_train, X_val, X_test, y_train, y_val, y_test
```

Output: train (35000,), test (10000,), valid (5000,), which is the 7:1:2 split. stratify keeps the positive/negative balance equal in all three sets.

(b)

```
def build_vocab(x_train: list, min_freq: int = 5, hparams=None) -> dict:
    if hparams is None:
        hparams = HyperParams()

    corpus = Counter()
    for review in x_train:
        corpus.update(t for t in review.split() if t not in hparams.STOP_WORDS)

    corpus_ = [word for word, freq in corpus.items() if freq >= min_freq]
    vocab = {w: i + 2 for i, w in enumerate(corpus_)}  # 0 and 1 are reserved
    vocab[hparams.PAD_TOKEN] = hparams.PAD_INDEX
```

```
    vocab[hparams.UNK_TOKEN] = hparams.UNK_INDEX
    return vocab
```

Word counts come from the training set only, so no information leaks from validation or test. Output: vocabulary size 54,327.

(c)

```
def tokenize(vocab: dict, example: str) -> list:
    unk_index = vocab['<unk>']
    return [vocab.get(word, unk_index) for word in example.split()]
```

Words missing from the vocabulary (stop words and rare words) map to <unk>, so the output length equals the number of words.

(d)

getitem tokenizes the review into ids, truncates to max_length, and converts the string label to a binary index.

```
def __getitem__(self, idx: int):
    ids = tokenize(self.vocab, self.x[idx])[:self.max_length]  # truncate to max_length
    length = len(ids)
    label = 1 if self.y[idx] == 'positive' else 0
    return {'ids': ids, 'length': length, 'label': label}
```

(e)

The model has an embedding layer, an LSTM, a dropout layer, and a fully-connected layer.

```
class LSTM(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim,
                 n_layers, dropout_rate, pad_index, bidirectional, **kwargs):
```

```
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=pad_index)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, num_layers=n_layers,
                            bidirectional=bidirectional, batch_first=True)
        self.fc = nn.Linear(hidden_dim * (2 if bidirectional else 1), output_dim)
        self.dropout = nn.Dropout(p=dropout_rate)

        # Weight initialization. DO NOT CHANGE!
        if "weight_init_fn" not in kwargs:
            self.apply(init_weights)
        else:
            self.apply(kwargs["weight_init_fn"])

    def forward(self, ids, length):
        embedded = self.dropout(self.embedding(ids))
        packed = nn.utils.rnn.pack_padded_sequence(
            embedded, length.cpu().long(), batch_first=True, enforce_sorted=False)
        _, (hidden, _) = self.lstm(packed)

        if self.lstm.bidirectional:
            final = torch.cat((hidden[-2], hidden[-1]), dim=1)
        else:
            final = hidden[-1]

        return self.fc(self.dropout(final))  # logits: [batch, output_dim]
```

(f)

Epoch	Train loss	Valid loss	Valid acc
1	0.689	0.660	0.625
2	0.471	0.397	0.829
3	0.270	0.387	0.853

Epoch	Train loss	Valid loss	Valid acc
4	0.169	0.364	0.861
5	0.111	0.480	0.867

I kept the epoch-4 weights (lowest validation loss) for testing: test loss 0.365, test accuracy 0.857.

Learning dynamics.

- **Training loss** falls at every epoch, from 0.689 (about ln 2, random guessing) to 0.111. Epoch 1 is slow partly because of the 200-step learning rate warmup.
- **Validation loss** is not steady. It falls until epoch 4 (0.364), then rises to 0.480 while training loss keeps falling. This is overfitting. The model memorizes the training reviews and becomes overconfident on its mistakes. Validation accuracy still edges up, so the increase comes from a few confidently wrong predictions.

First test example. The review (about the Dr. Seuss estate) is truly negative. The model predicted negative with probability 0.867. This meets my expectation, since test accuracy is 85.7%. The confidence is moderate because each word is a single number (`EMBEDDING_DIM = 1`) and the review expresses negativity as a complaint, not through strongly negative words. One example is only anecdotal.

3 Lab (2) Large Language Model for Text Generation (45 pts)

(a)

```
model = AutoModelForCausalLM.from_pretrained("gpt2").to(device)
tokenizer = AutoTokenizer.from_pretrained("gpt2")
tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = model.config.eos_token_id

inputs = tokenizer(prompt, return_tensors="pt").to(device)
gen_tokens = model.generate(input_ids=input_ids, attention_mask=attention_mask, max_length=max_length)
gen_text = tokenizer.decode(gen_tokens[0], skip_special_tokens=True)
```

Output (pre-trained): GPT-2 is a language model based on transformer developed by OpenAI. It is a language that is used to describe the behavior of a system. It is a language that is used to describe the behavior of a system. ...

(b)

```
dataset = load_dataset("Salesforce/wikitext", "wikitext-2-raw-v1")

def tokenize_function(examples):
    tokenized = tokenizer(examples["text"], truncation=True, max_length=128, padding="max_length")
    tokenized["labels"] = tokenized["input_ids"].copy()
    return tokenized

tokenized_datasets_train = dataset_train.map(tokenize_function, batched=True, remove_columns=["text"])
tokenized_datasets_valid = dataset_valid.map(tokenize_function, batched=True, remove_columns=["text"])
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)
```

Batch shape: [4, 128] for input_ids, attention_mask and labels.

(c)

```
input_ids = batch["input_ids"].to(device)
attention_mask = batch["attention_mask"].to(device)
labels = batch["labels"].to(device)

outputs = model(input_ids=input_ids, attention_mask=attention_mask)
logits = outputs.logits

loss = loss_fn(shift_logits.view(-1, shift_logits.size(-1)), shift_labels.view(-1))
```

Initial perplexity (pre-trained): 48.56

(d)

```
training_args = TrainingArguments(
    output_dir="./gpt2-wikitext-2",
    num_train_epochs=3,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    eval_steps=400,
    save_steps=800,
    warmup_steps=500,
    prediction_loss_only=True,
    eval_strategy="epoch",
    logging_strategy="epoch",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets_train,
    eval_dataset=tokenized_datasets_valid,
    data_collator=data_collator,
)
result_full = trainer.train()
trainer.save_model()

model_finetuned = AutoModelForCausalLM.from_pretrained("./gpt2-wikitext-2").to(device)
```

Epoch	Training loss	Validation loss
1	3.758	3.340
2	3.173	3.377
3	2.872	3.409

Training loss keeps falling, but validation loss is lowest at epoch 1 and then rises. The model starts to overfit the small training subset after the first epoch.

Fine-tuned perplexity: 29.82

Generated text: GPT-2 is a language model based on transformers developed by OpenAI , a consortium of universities and universities . The language model is based on the concept of a set of functional languages , which are used to represent the language . ...

(e)

```
model_lora = AutoModelForCausalLM.from_pretrained("gpt2")
model_lora = get_peft_model(model_lora, peft_config).to(device)

# TrainingArguments: same as (d), with output_dir="./gpt2-lora-wikitext-2"
trainer = Trainer(
    model=model_lora,
    args=training_args,
    train_dataset=tokenized_datasets_train,
    eval_dataset=tokenized_datasets_valid,
    data_collator=data_collator,
)
result_lora = trainer.train()
```

Epoch	Training loss	Validation loss
1	4.304	3.735
2	3.925	3.546
3	3.826	3.518

Trainable parameters: 2.36M of 126.8M (1.86%).

LoRA perplexity: 32.89

Generated text: GPT-2 is a language model based on transformers developed by OpenAI , which is based on the concept of a "transformation-based" language model . The model is based on the concept of a "transformation-based" language model , ...

(f)

```
print(result_full.metrics["train_runtime"], result_lora.metrics["train_runtime"])
```

Model	Training time	Perplexity
Pre-trained GPT-2	none	48.56
Fully fine-tuned	7 min 20 s	29.82
LoRA fine-tuned	5 min 19 s	32.89

Training time. Full fine-tuning took 7:20 and LoRA took 5:19, so LoRA was about 28% faster. LoRA trains only 1.86% of the weights. It does not compute weight gradients or optimizer updates (Adam) for the other 98%, so each step is cheaper and needs less memory. The saving is moderate because the forward pass and the backward pass through the network are still full size.

Perplexity. Pre-trained 48.56, full fine-tuning 29.82, LoRA 32.89. Both fine-tuned models improve a lot because they adapt to Wikipedia text. Full fine-tuning is best because it can change all weights. LoRA can only change a small low-rank part of the model, so it fits the data a bit less well. Full fine-tuning also overfits (validation loss rises after epoch 1). Its perplexity would likely be a little better with fewer epochs. LoRA's validation loss kept falling.

Generated text. All three outputs repeat themselves. This is typical of greedy decoding with a small model. The pre-trained text is plain prose and loops on "a language that is used to describe the behavior of a system". Both fine-tuned models copy the WikiText format (spaces before commas and periods) and sound more encyclopedic. Their content is still wrong, for example "a consortium of universities". So the lower perplexity shows better style and word statistics, but not better factual quality.